

编者按：这是一份完整的项目总结文件 PRJ_SUM_0426.md，它涵盖我从数据准备到模型优化探索的全部工作，并提出后续的行动方向，敬请审阅。

鑫光板材瑕疵检测项目总结报告

文档编号：PRJ_SUM_0426.md

撰写日期：2026年4月26日

项目负责人：彭日骏、刘宇杭、袁康睿

项目背景：英特尔杯嵌入式 AI 专题赛，基于 DK2500 开发套件的工业板材表面瑕疵检测系统。

一、项目目标

利用鑫光板材数据集，训练一个 YOLOv8 目标检测模型，识别三类板材表面瑕疵，并在目标硬件 DK2500（Intel Core Ultra 5 225U，NPU 加速）上实现实时推理。

瑕疵类别：

- defect_2：点状小瑕疵（如针孔、气泡）
- defect_3：中型区域瑕疵（如划痕、凹陷）
- defect_6：大型或边缘瑕疵

二、已完成工作总览

2.1 数据工程

- 原始数据：**工业线扫长图（分辨率 1326×5006），LabelMe JSON 矩形框标注。
- 数据源：**D:\dataset_2\xinguang\defect_detection\dataset\defects，仅处理瑕疵检测数据集（不包含圆孔检测与尺寸检测数据）。
- 预处理脚本：**scripts/prepare_dataset.py，实现以下功能：
 - 滑窗切割：**窗口 640×640，步长 500，首窗从原点开始，末端自动贴边，确保图像边缘区域完整覆盖。对于不足 640×640 的边缘窗口，采用黑色像素补齐。

- **标注转换与保留判定**：LabelMe JSON → YOLO TXT 格式。对每个原图标注框与裁剪窗口计算交集，采用“双重保留机制”：交集面积占原框面积 > 20%，或者交集绝对面积 > 200 像素。此举确保微小瑕疵即使被切到边缘也不会丢失。
- **负样本保留机制**：裁剪窗口内无任何有效标注框时，按 10% 概率随机保留该子图，仅生成图片文件，不生成对应的 TXT 标注文件。负样本在后续划分中随正样本一起随机打乱，最终同时分布于训练集与验证集，以便在评估阶段检测模型的误报率。
- **数据集划分**：将所有样本（含正负）随机打乱后，按 8:2 比例划分训练集与验证集（随机种子固定为 42，确保可复现）。
- **最终数据集**：训练集约2630 张，验证集约658张 张。类别数 3（defect_2/3/6）。
- **数据集配置**： configs/xinguang.yaml，由脚本自动生成，包含训练/验证集绝对路径与类别名称映射。

2.2 模型训练与实验

以下为本项目已完成的三轮完整训练：

实验编号	训练名称	模型配置	轮次	最终 mAP50	备注
Exp-01	baseline_train	YOLOv8n， 默认参数	50	0.5855	基线模型， 当前最优
Exp-02	cbam_wiou_v1	YOLOv8n + CBAM 注意力 + WIoU 损失	100	0.5711	魔改实验， 未超越基线
Exp-03	baseline_aug_v1	YOLOv8n，降 mosaic、加 copy_paste	50	0.4466	数据增强调整， 精度大幅倒退

Exp-02 归因分析：

- CBAM 的**全局池化机制**会系统性地淹没微小瑕疵的特征信号。
- CBAM 插入位置（Head 的 P3 层）对小目标不利，P3 层特征图空间分辨率不足。
- WIoU 默认超参数（ $\gamma=1.9$ ）针对 COCO 大规模数据集调优，在小数据集上引发收敛困难。
- CBAM 的随机初始化权重对预训练模型造成了破坏性干扰。

Exp-03 归因分析：

- 将 mosaic 从 1.0 骤降至 0.3，导致训练样本多样性断崖式下跌。
- 在未同步调整学习率和训练轮次的情况下，模型收敛受阻。

2.3 部署验证

- 推理加速：成功将 Baseline 模型导出为 OpenVINO 格式，在 Intel Core Ultra 7 155H（纯 CPU）上实现：
 - 纯推理 FPS：**107.32**。
 - 平均单张推理耗时：**9.32 ms**。
- 结论：部署链路完全打通，推理速度远超 DK2500 实时检测需求（目标 FPS ≥ 50 ）。

2.4 官方模型对比测试

为证明定制训练的必要性，对官方 yolov8n.pt 与本项目 Baseline best.pt 进行了精度与速度双维度对比：

模型	mAP50	OpenVINO FPS
官方 YOLOv8n	0.005	~13
Baseline (Exp-01)	0.5855	~107

官方模型在本任务上完全失效（精度趋近于零），证明了针对特定工业场景进行定制训练的不可替代性。

2.5 项目文件结构

```
XinGuang/
├── scripts/                                # 可执行 Python 脚本
│   ├── prepare_dataset.py                # 数据预处理脚本
│   ├── start_training.py                 # 模型训练启动脚本
│   ├── train_v2.py                       # 实验版训练脚本
│   ├── resume_training.py                # 断点续训脚本
│   ├── val_baseline.py                   # PyTorch 精度验证脚本
│   ├── infer_openvino.py                 # 单模型 OpenVINO 推理与速度测试
│   └── full_compare_openvino.py          # 双模型 OpenVINO 全面对比脚本
├── configs/                               # 配置文件
│   ├── xinguang.yaml                     # 数据集配置
│   ├── yolov8n_cbam_wiou.yaml            # CBAM+WIoU 魔改模型配置（已废弃）
│   └── yolov8n-p2.yaml                   # 纯净版 P2 检测头模型配置（待验证）
├── docs/                                  # 文档
│   ├── PROJECT_SUMMARY.md                # 早期项目总结
│   ├── SUM_0426.md                       # 现在这份报告
│   └── 2026年4月22日阶段性报告.md       # 4月22日刚跑完实验3时候的报告
├── datasets/                              # YOLO 格式数据集
│   ├── images/train/                     # 训练图片（3287张）
│   ├── images/val/                       # 验证图片（822张）
│   ├── labels/train/                     # 训练标注
│   └── labels/val/                       # 验证标注
├── runs/                                  # 训练结果与推理输出
│   └── detect/
│       ├── baseline_train/               # Exp-01 Baseline 训练结果（含 best.pt）
│       ├── cbam_wiou_v1/                 # Exp-02 魔改训练结果
│       ├── baseline_aug_v1/              # Exp-03 数据增强调整结果
│       └── ...                           # 其他推理输出目录（不重要了。）
├── requirements.txt                       # 依赖清单
└── README.md                             # 项目说明
```

三、当前精度瓶颈分析（基于最初的实验baselian模型的分析，不涉及两次优化的尝试）

3.1 各类别精度对比（跑python scripts\val_baseline.py得到的）

类别	描述	Baseline AP50	问题定性
defect_2	点状小瑕疵	0.417	核心短板
defect_3	中型瑕疵	0.689	已接近可用水平
defect_6	大型/边缘瑕疵	0.653	已接近可用水平

3.2 瓶颈原因分析与推测

defect_2 小目标漏检是限制整体精度的主要核心矛盾。 原因分析如下：

- 1. 物理像素瓶颈：YOLOv8n 默认最小检测头 P3 下采样率为 8 倍。10×10 像素的 defect_2 在 P3 层仅剩约 1.25×1.25 像素，特征在物理层面几近消失。
- 2. 背景干扰：复杂木纹纹理与小瑕疵在特征上高度相似，模型难以区分。

四、下一步工作建议（三次训练实验表明，单靠我的电脑CPU太慢了，跑不动了，要跑好久，所以由刘宇杭继续吧）

4.1 最高优先级：P2 检测头验证

目标：在不改动其他参数的前提下，仅增加 P2 高分辨率检测头，验证其对 defect_2 的提升效果。

操作步骤：

- 1. 确认配置文件 configs/yolov8n-p2.yaml 已就绪。
- 2. 修改 scripts/start_training.py ，将模型加载路径改为 configs/yolov8n-p2.yaml ，输出名称改为 baseline_p2_v1 。
- 3. 运行 python scripts/start_training.py ，训练 50 轮。
- 4. 验证标准：50 轮后 defect_2 的 AP50 > 0.50、整体 mAP50 > 0.60，则路线走通。

预期收益：defect_2 AP50 提升 10~15 个百分点，整体 mAP50 冲击 0.62~0.65。

预期耗时：纯 CPU 约 8 小时 / 50 轮，GPU可能减半吧。

4.2 DK2500 部署验证

目标：验证模型在目标硬件上的推理速度。

操作步骤：

- 1. 取 Baseline best.pt （或 P2 模型 best.pt ，如果 4.1 已验证成功）。
- 2. 导出为 OpenVINO 格式：

```
from ultralytics import YOLO
model = YOLO('runs/detect/baseline_train/weights/best.pt')
model.export(format='openvino')
```

- 3. 在 DK2500 上运行 OpenVINO 推理，确认 FPS ≥ 50。

4.3 若 P2 路线验证成功——申请云端 GPU 进行充分训练

目标：将 P2 模型跑满 300 轮，充分收敛，冲击更高精度。

预期收益：mAP50 冲击 0.75~0.78。

建议平台：AutoDL（租用 RTX 4090，约 1.5-3 元/小时），预计 300 轮总耗时 3-5 小时，总费用 10-15 元。

4.4 其他可选优化方向

以下方向可在 P2 路线确认净收益后，作为叠加实验逐一验证：

优先级	优化项	预期收益	风险
中	余弦退火学习率 （ cos_lr=True ）	+1~2 点	低
中	K-means 定制锚框	+2~3 点	低
低	CoordAtt 注意力模块	+3~5 点	中等，需验证 OpenVINO NPU 兼容性
不推荐	CBAM 注意力	—	已证伪，对小目标有反效果
不推荐	WIoU 损失函数	—	小数据集上已证实水土不服

五、核心经验教训（实践证明了的）

1. **“教条主义”不可取**：方案书中原先由AI预设的 CBAM+WIoU 路线，在未经任务适配性审查的情况下直接投入全量训练，导致 100 轮后精度反降。
2. **低成本验证是生命线**：任何模型结构或超参数改动，必须先通过 20-30 轮快速消融实验观察趋势，再决定是否投入完整训练。
3. **工业场景需要考虑硬件部署约束**：模块选型必须同时满足“精度有效”与“OpenVINO NPU 兼容”两个条件。非标准算子可能导致推理速度断崖式下跌。
4. **负样本保留机制是关键**：在数据预处理阶段保留无瑕疵背景图，是防止产线误报的基础保障。

同志们，我的报告就到这里。

雄关漫道真如铁，而今迈步从头越！